MCP A2A Bridge Policy

MCP A2A Bridge Policy

A custom Omni Gateway (Flex Gateway) policy that lets MCP clients use an A2A agent broker, such as a MuleSoft Agent Broker, as if it were an MCP server.

Exchange asset	`mcp-a2a-bridge-policy` (definition), `mcp-atoa-bridge-policy-flex` (implementation)
Current version	1.1.0
Category	Transformation
Applies to API instance types	MCP, A2A (v0.3), A2A (v1.0)
Gateway	Omni Gateway / Flex Gateway (managed or self-managed)

1. Purpose

The policy turns an A2A agent broker into an MCP server with a single tool (`ask_agent` by default). MCP clients find the tool, call it with a natural-language question, and get the broker's answer back as an ordinary MCP tool result. The broker itself needs no changes.

2. Problem statement

- **Agent brokers speak only A2A.** A MuleSoft Agent Broker accepts A2A requests (`SendMessage`, tasks, agent card).
- **Many AI clients speak only MCP.** ChatGPT, Claude, Cursor and Copilot connect to MCP servers. They can't call an A2A endpoint.
- **The two protocols differ.** They use different messages, different conversation handling (A2A context and task IDs versus MCP tool calls), and different response shapes.

Without the policy, every team that wants these clients to reach the broker has to build and run its own MCP wrapper service. The policy does the conversion inside the gateway instead, alongside the existing security, logging and rate-limiting policies, and needs no extra service.

3. How it works

MCP client sends	Policy does
`initialize`, `ping`, `tools/list`, notifications	Answers them at the gateway; nothing reaches the broker
`tools/call` for `ask_agent`	Converts the call into an A2A `SendMessage`, forwards it to the broker, and converts the A2A reply back into an MCP tool result

MCP client sends	Policy does
Native A2A request (Mode 2 only): JSON-RPC (v1.0 or v0.3), HTTP+JSON (`/message:send` , `/tasks/{id}:cancel` , …), agent card, gRPC	Forwards it to the broker unchanged: same path, headers and body

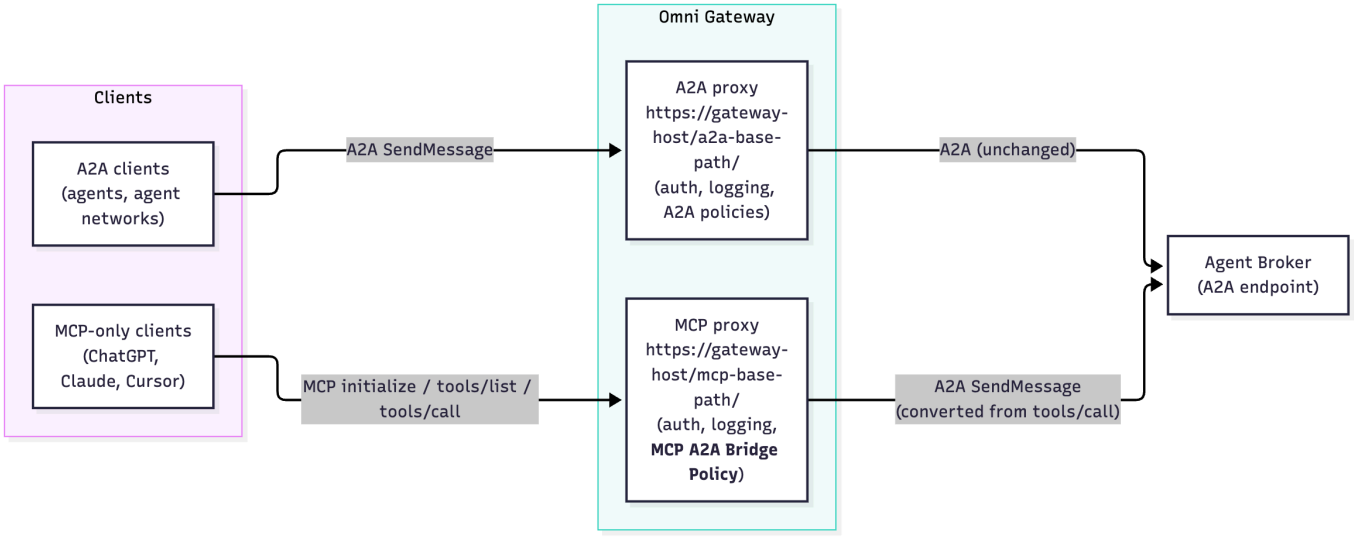
Multi-turn conversations. Every tool result returns the broker's `contextId` , `taskId` and `state` . - To **continue a conversation**, the client passes `context_id` back. - To **answer a question the broker asked** (state `input-required`), the client passes `context_id` and `task_id` .

---

4. Deployment modes

Mode 1: separate MCP proxy next to the A2A proxy

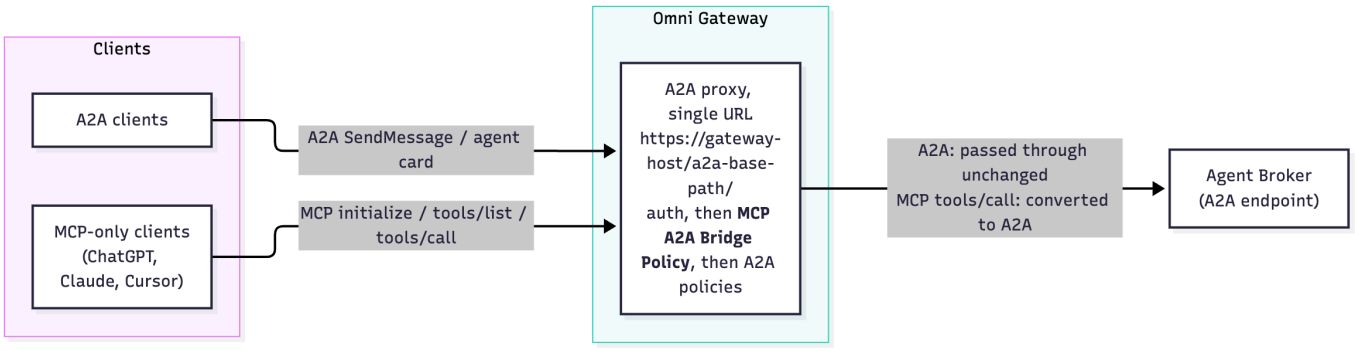
The broker keeps its existing A2A proxy. A second API instance of type **MCP** points to the same broker, and the policy is applied **only on the MCP proxy**. A2A clients and MCP clients each use their own URL.



- **When to use:** you want separate URLs, policies, contracts and analytics for MCP and A2A traffic. This is the recommended setup.
- **Key settings:** `passThroughA2A: false` . Use `a2aPath: "/"` so MCP clients can call either `<mcp-base-path>/` or `<mcp-base-path>/mcp` .

Mode 2: one proxy for both A2A and MCP clients

Only the broker's A2A proxy exists, and the policy is applied **directly on that A2A proxy**. A2A clients and MCP clients use the **same URL**.



How the policy decides what to do with each request: 1. **The request is MCP**, for example `initialize`, `tools/list` or `tools/call`. The policy answers it itself or converts it, as in section 3. 2. **The request is native A2A**, for example a `SendMessage` or `message/send` JSON-RPC call, an HTTP+JSON call such as `POST /message:send`, or the agent card. With `passThroughA2A: true`, the policy recognises it before any MCP handling and forwards it to the broker untouched. It does not add or change the `A2A-Version` header, the path or the body, and the broker's response goes back unchanged.

Only translated `tools/call` requests are changed. Their `A2A-Version` header and query parameter always match the configured `a2aProtocolVersion`: `1.0` sets the header to `1.0`, and `0.3` removes it.

- **When to use:** you want a single endpoint for every client, or you can't create a second API instance.
- **Key settings:** `passThroughA2A: true` and `a2aPath` left empty. MCP clients call the A2A base URL, with the trailing slash.
- **Policy order:** put the bridge **after** authentication policies (client ID, OAuth/OIDC) and **before** A2A-specific policies (A2A schema validation, A2A PII detection). Those A2A policies then see the converted A2A message.

---

5. Configuration properties

Property	Required	Default	Description	Sample value
`toolName`	Yes	`ask_agent`	Name of the MCP tool shown to clients. Letters, digits, `_` and `-`; up to 64 characters.	`ask_hr_agent`
`toolDescription`	No	Generic description	Text the client's AI model reads to decide when and how to call the tool. **This is the most important setting for answer quality** (see 5.1).	`Ask the <Org> <Domain> agent network about <topics>. HOW TO CALL: (1) ... (2) ... (3) ...`
`serverName`	No	`mcp-a2a-bridge`	Server name returned to MCP clients during the handshake.	`<org>-hr-assistant`

Property	Required	Default	Description	Sample value
`serverInstructions`	No	empty	Short usage instructions returned during the handshake. Not every client uses them.	`Use ask_agent. Reuse context_id for follow-ups and restate the subject in full.`
`a2aProtocolVersion`	Yes	`1.0`	A2A version the broker uses: `1.0` or `0.3`.	`1.0`
`passThroughA2A`	No	`false`	`false`: accept MCP traffic only (Mode 1). `true`: also forward native A2A traffic unchanged (Mode 2).	Mode 1: `false` / Mode 2: `true`
`a2aPath`	No	empty	Path on the upstream (broker) URL that converted calls are sent to. Empty forwards the path the client used. `/` sends every call to the broker's root URL. Native A2A traffic is never rewritten.	Mode 1: `/` / Mode 2: empty / custom: `/<a2a-sub-path>`
`includeStructuredContent`	No	`true`	Adds `contextId`, `taskId` and `state` to each tool result. Keep it on so clients can hold multi-turn conversations.	`true`
`sessionStoreMode`	No	`disabled`	Optional gateway-side memory of the last conversation per MCP session, for clients that never send `context_id` back. `local` keeps it in memory on each gateway replica; `remote` needs gateway Shared Storage (Redis).	`disabled`
`sessionTtlSeconds`	No	`1800`	How long, in seconds, a remembered conversation lasts (60–86400). Used only with a session store.	`1800`

Property	Required	Default	Description	Sample value
`sessionKeyHeader`	No	empty	Request header whose value is mixed into the session key, so a remembered conversation is tied to the caller. When set, this is enforced: a request without the header gets no session and can't use or end one. Used only with a session store.	`client_id`

Limits and session security (1.1.0) - **Response size.** The gateway can't write a body of 1 MB or more. If a converted answer would be too large, the policy first drops `structuredContent.data` (data parts are also in the text), then truncates the answer text with a note, and as a last resort returns a tool error. The client always gets a valid MCP result, never the raw A2A reply. The remembered conversation is updated only after the answer is delivered. A `tools/call` message too large to forward gets error `-32602`. - **Sessions.** `Mcp-Session-Id` values are random UUIDs, issued at `initialize` and registered in the store. The policy ignores any other value a client sends. Store keys are a SHA-256 hash of the API/policy scope, session ID and identity. The store namespace is specific to the API instance and policy, so two APIs that share one Redis don't share sessions, while replicas of the same API do. Expiry uses the TTL configured now, so a lower `sessionTtlSeconds` also shortens existing entries. `DELETE` with `Mcp-Session-Id` returns `204` for the owner and `404` otherwise.

5.1 Writing a good `toolDescription`

The broker doesn't remember earlier answers across separate questions. The description should therefore tell the client's AI model: 1. **What the agent covers**, for example the domains and agents behind the broker. 2. **New topic:** send only the message. 3. **`input-required`** (the broker asked a question): reply with the same `context_id` **and** `task_id`. 4. **Follow-up after a completed answer:** send the same `context_id`, with no `task_id`, and **restate the subject in full** (names, IDs, record numbers). For example, write "What is Jane Doe's email?" rather than "What is her email?". 5. **Never invent IDs.**

The API Manager form shows this field on a single line, so write the rules inline: `(1) ... (2) ... (3) ...`.

5.2 Recommended policy order

Order	Mode 1: MCP proxy	Mode 2: A2A proxy
1	Logging / tracing (optional)	Agent card, logging, tracing (existing)
2	OAuth 2.0 Protected Resource Metadata (if you use OAuth)	Authentication (Client ID enforcement / OIDC)
3	Authentication (OIDC access token or Client ID enforcement)	**MCP A2A Bridge Policy**
4	**MCP A2A Bridge Policy**	A2A policies (PII detection, schema validation)
5	Response timeout (recommended: 120 s)	

- **Authentication must run before the bridge.** The bridge answers `initialize` and `tools/list` itself, so any policy placed after it never runs for those requests.
- **Raise the response timeout.** Questions that span several systems can take 30–60 s, and the default MCP timeout is shorter.

---

6. Endpoints to share with clients

Mode	Client	URL
Mode 1	MCP clients	`https://<gateway-host>/<mcp-base-path>/` or `https://<gateway-host>/<mcp-base-path>/mcp`
Mode 1	A2A clients	`https://<gateway-host>/<a2a-base-path>/`
Mode 2	MCP and A2A clients	`https://<gateway-host>/<a2a-base-path>/` (trailing slash required)

- **Clients that can't send custom headers** (ChatGPT, for example) can't pass `client_id` and `client_secret`, so protect the endpoint with OAuth 2.0. Apply the OAuth 2.0 Protected Resource Metadata policy and an OIDC access token policy.
- **Registering the proxy as an MCP asset in Exchange:** the tool definition (`ask_agent`, with inputs `message`, `context_id` and `task_id`) can be published in the MCP schema.

---

7. Publishing the policy to Anypoint Exchange

The project uses the PDK single-project layout. The definition (`definition/gcl.yaml`) and the implementation (`src/`) live in one project, and one command publishes both.

```
mcp-a2a-bridge-policy/
├── .project.yaml       # PDK project descriptor
├── Makefile
├── Cargo.toml          # crate and Exchange coordinates
├── definition/gcl.yaml # configuration schema
└── src/                # implementation
```

7.1 Prerequisites

- **Anypoint CLI v4** with the PDK plugin (`anypoint-cli-v4 plugins:install anypoint-pdk-plugin`), plus the Rust toolchain. For first-time setup, run `make setup`.
- **A connected app** (client credentials) with the **Exchange Contributor** permission in the target business group.

7.2 Set the target business group and version

`Cargo.toml`

```toml
[package]
name = "mcp_a2a_bridge_policy"
version = "<x.y.z>"

[package.metadata.anypoint]
group_id = "<business-group-id>"
definition_asset_id = "mcp-a2a-bridge-policy"
implementation_asset_id = "mcp-atoa-bridge-policy-flex"
```

- **One version for both assets.** The definition and the implementation are published with the `version` in `Cargo.toml`.
- **Released versions are permanent.** Exchange won't overwrite a released version, so bump the version for every change.
- **Implementation asset ID.** It must not contain digits next to letters. PDK kebab-cases the ID, so `a2a` becomes `a-2-a` and the ID is rejected.

7.3 Credentials (environment variables)

```bash
export ANYPOINT_CLIENT_ID=<connected-app-client-id>
export ANYPOINT_CLIENT_SECRET=<connected-app-client-secret>
export ANYPOINT_ORG=<business-group-id>
```

7.4 Publish

Command	Result
`make release`	Builds the policy and publishes the definition and implementation as version `<x.y.z>`
`make publish`	Publishes a **development** version (`<x.y.z>-DEV`) for testing

Run either command from the project root.

7.5 Verify

1. **Exchange:** in the business group, both `mcp-a2a-bridge-policy` and `mcp-atoa-bridge-policy-flex` show version `<x.y.z>`.
2. **API Manager:** open an MCP or A2A API instance, then Policies → Add policy. **MCP A2A Bridge Policy** appears under **Transformation**.